

Modul Praktikum

IF302 SISTEM OPERASI

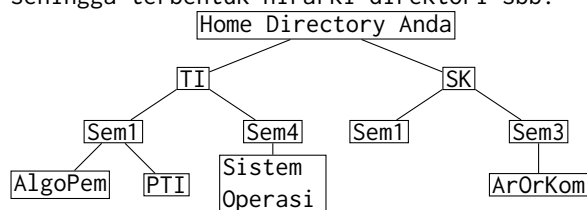


Tujuan Praktikum 1.1

1. Mhs mengerti Sistem Operasi linux, perintah dasar pada console linux

Petunjuk Praktikum 1.1

1. Bukalah console (terminal program)
2. Untuk ke home directory ketik **cd** atau **cd** kemudian tekan enter
3. Lihatlah home directory anda dengan command **pwd**
4. Untuk mengetahui login name gunakan command : **whoami**
5. Untuk mengetahui versi kernel gunakan command : **uname -r**
untuk option-option yang lainnya gunakan **uname --help**
6. Melihat jam/tanggal gunakan command : **date**
7. Melihat kalender gunakan command: **cal**
8. Melihat user yang saat ini sedang login : **who**
9. Melihat login terminal : **tty**
10. Melihat isi text file per halaman : **more <namafile>**
11. Melihat jumlah baris, kata dan karakter dari suatu text file :
wc <namafile>
12. Melihat jumlah baris saja : **wc -l <namafile>**
13. Melihat jumlah kata saja : **wc -w <namafile>**
14. Melihat jumlah karakter saja : **wc -c <namafile>**
15. Dari home directory anda cobalah buat folder/direktori sehingga terbentuk hirarki direktori sbb:



Untuk membuat direktori gunakan command **mkdir** sbb: **mkdir <nama direktori>**

16. Setelah hirarki direktori terbentuk cobalah pindah-pindah direktori dengan command **cd** (change directory (**cd <direktori>**), dengan menggunakan absolute path dan relative path.
17. Untuk kembali ke direktori sebelumnya : **cd -**
18. Untuk naik satu tingkat ke direktori di atasnya : **cd ..**
19. Bukalah salah satu Text editor, kemudian ketik program dibawah ini.

```
#include <stdio.h>
int main()
{
    printf("Selamat datang di laboratorium perangkat lunak - UMN\n");
    printf("Semoga senang belajar Sistem Operasi Linux\n");
    return 0;
}
```

Kemudian Save As dengan nama file **PrakOS1.c** pada home directory anda, kemudian close text editor nya.

20. Copy file **PrakOS1.c** pada home directory anda ke directory "Sistem Operasi", dengan perintah **cp**. (**cp <source> <target>**)
21. Pindahkan file **PrakOS1.c** yang ada di home directory dengan perintah **mv** (move/rename) ke directory **/Sem1/AlgoPem**

22. Rename (ganti nama) file **PrakOS1.c** yang ada di directory AlgoPem dengan nama **AlgoPem1.c** dengan perintah **mv** (move/rename)
23. Remove / hapus file **AlgoPem1.c** dengan perintah **rm** (rm <namafile>)
24. Hapus direktori AlgoPem dengan command : **rmdir**
25. Untuk melihat isi direktori gunakan command : **dir**, **ls** atau **ls -l**
26. Kompilasi program **PrakOS1.c** yang ada di direktori **Sistem Operasi** dengan perintah sbb:
gcc -o PrakOS1 PrakOS1.c
kemudian jalankan program tersebut dengan perintah :
./PrakOS1 <enter>
27. Coba lihat isi text file **PrakOS1.c** dengan perintah **cat** (cat <namafile>) atau untuk menampilkan dengan nomor baris gunakan option **-n** (**cat -n PrakOS1.c**)
28. Buatlah sebuah file baru dan kosong dgn nama **CobaTouch.txt** dengan command **touch** (**touch CobaTouch.txt**), kemudian lihat nama file dan isinya.
29. Copy direktori berserta isinya dgn command : **cp -r <source dir> <new dir>**
30. Coba Rename direktori dengan command **mv**
31. Hapus direktori beserta isinya : **rm -fr <direktori>**
32. Untuk melihat dokumentasi perintah / command gunakan **man** <perintah yg ingin dilihat> (contoh man ls)
33. Membuat file text dengan command **cat** :
cat > namafile.txt <enter>
<masukan text file> <enter>
<CTRL-D>
34. Mengubah working direktori ke /sem3/ArOrKom, copy beberapa file dari sembarang direktori ke direktori ini, kemudian buat file archive (file.tar) dengan command : **tar -cvf <nama file.tar>**.
35. Isi dari file tar dapat dilihat dgn command : **tar -tf <namafile.tar>**
36. Buatlah direktori pada tempat dan nama sembarang, kemudian copy file .tar ini ke direktori tersebut, kemudian ekstraklah file tersebut dengan command : **tar -xvf <namafile.tar>**
37. Lakukan kompresi dari file .tar tadi dengan command :
gzip <file.tar> <nama-nama file yg dikompres, dipisahkan dgn spasi>
38. Untuk membuat kompresi file.tar.gz dengan command : **tar -zcvf <archive-name.tar.gz> <source-folder-name>**
39. Untuk membuka/ekstak file.tar.gz commandnya : **gunzip <file.tar.gz>** atau bisa juga menggunakan command : **tar -zxvf <archive-name.tar.gz >**
40. Cek sistem operasi linux yang digunakan apakah 32-bit atau 64-bit dengan command : **uname -m** (jika keluar x86_64 berarti 64 bit)

Tujuan Praktikum 1.2

1. Mhs mengerti perintah dasar editor vi / (VIM - Vi IMproved)

Petunjuk Praktikum 1.2

1. Bukalah console (terminal program)
2. Jalankan editor vi dengan command: **vi**
i = insert mode
ESC = command mode

x = delete char
dd = delete line
J = delete line break
u = undo
a = append
o = insert line di bawah cursor
O = insert line di atas cursor
:w <namafile> = simpan dengan namafile
:set number
:set nonumber
:/string => mencari string
xyy = copy x line ke bawah -> x = jumlah line
v = block kata / kalimat
p = paste
command-command yang lain silahkan baca di :
/usr/share/vim/vim64/doc/usr_xx.txt

3. Buatlah program dibawah ini dengan editor vi dengan nama file **testvi.c** (vi **testvi.c** <enter>)

```
#include <stdio.h>
int main()
{
    printf("Program ini diedit dengan editor vi / VIM\n");
    return 0;
}
```

kemudian kompilasi, dan coba jalankan!

4. Disamping editor vi, pada console juga kadang-kadang ada editor **pico**, **nano** silahkan dicoba untuk membuat program seperti diatas.
(Catatan: Editor-editor ini sering diperlukan oleh seorang admin (super user), untuk konfigurasi sistem).

Petunjuk Praktikum 1.3

- Sebelum keluar ruangan simpanlah history dari command-command yang telah anda buat ke text file.
Buat Nama file sama dengan **NIM_History.txt**, Contoh dengan command:
history > 12345698_History.txt
- Kemudian jawab pertanyaan dibawah ini dengan nama file **NIM_Jawab.txt**
 - Jelaskan perbedaan antara root directory, home directory dan working directory
 - Jelaskan perbedaan antara absolute path dengan relative path
- Kompres kedua file diatas (NIM_History.txt dan NIM_Jawab.txt) menjadi nama file **NIM_Nama.tar.gz**
Copy file tersebut dan dikumpulkan ke asisten.

Petunjuk Praktikum 1.4

- Buku console lagi dan login sebagai **root** (**sudo su**), kemudian isi password untuk **root**, maka prompt \$ akan berubah menjadi #
- Cobalah shutdown melalui console dengan perintah: **shutdown -h now**

Tujuan Praktikum 2.1

1. Mhs mengerti cara menambah user pada sistem operasi Linux

Petunjuk Praktikum 2.1

1. Buka console (terminal) kemudian tambahkan 1 user pada komputer Anda dengan nama login yang sesuai dengan nama Anda (satu kata saja).
2. Untuk menambahkan user gunakan command **sudo adduser <nama-login>**
<nama-login> adalah nama login (hanya satu kata saja).
3. CONTOH di Linux Ubuntu 14.02, jika nama login adalah **naruto** sbb:
sudo adduser naruto
[sudo] password for student: <masukkan password untuk root>
Adding user naruto ...
Adding new group 'naruto' (xxxx) ...
Adding new user 'naruto' (xxxx) with group 'naruto' ...
Creating home directory '/home/naruto' ...
Copying files from '/etc/skel' ...
Enter new UNIX password: <masukan password anda disini>
Retype new UNIX password: <masukan password sekali lagi>
passwd: password updated successfully
Changing the user information for naruto
Enter the new value, or press ENTER for the default
Full Name []: <Masukkan nama lengkap anda disini>
Room Number []:
Work Phone []:
Home Phone []:
Other []:
Is the information correct? [Y/n] Y
4. Kemudian pindah ke console2 dengan menekan (**CTRL + ALT + F2**) kemudian pada console ini silahkan login sebagai **student**. Kemudian buat sebuah text file dengan nama dan isinya sembarang.
5. Kemudian pindah ke text mode (console3) lainnya dengan menekan **CTRL+ALT+F3** kemudian pada console ini silahkan login sesuai dengan nama login Anda tadi. Kemudian buat sebuah text file dengan nama dan isinya sembarang.
6. Coba ketik command **who** pada console ini. Ada berapa user yang sedang login ?
7. Ada berapa terminal (console) yang sedang open ?
8. Jawaban pertanyaan diatas ditulis dalam sebuah text file dengan nama **jawab.txt** pada console3
9. Silahkan logout dari kedua console ini dengan command **exit**
Untuk kembali ke tampilan GUI tekan **CTRL + ALT + F7**

Tujuan Praktikum 2.2

1. Mhs mengerti dasar-dasar sistem file pada linux.
2. Mhs mengerti hak akses file dan direktori.

Petunjuk Praktikum 2.2

1. Silahkan gunakan console1 dengan menekan **CTRL + ALT + F1** dan login sebagai **student** dan console2 **CTRL + ALT + F2** sebagai user Anda sendiri.

2. Pada console2, buat file dengan nama **test.c** dengan menggunakan text editor **vi** atau **nano**. Isi dari file **test.c** adalah sbb:

```
#include <stdio.h>
int main()
{
    printf("Test...\n");
    return(0);
}
```
3. Setelah file diatas disimpan lihat hak akses file tersebut dengan command :
ls -l test.c
Jelaskan arti dari **rw-rw-r--** **test.c**
Jawaban ditulis di file **jawab.txt** diatas
4. Kemudian buat file **test.c** hanya bisa dibaca saja oleh pemiliknya dengan command :
chmod 400 test.c
5. Kemudian coba modifikasi file **test.c** di atas kemudian simpan, Apa yang terjadi?
Jelaskan dengan singkat saja!
6. Ubah hak akses file **test.c** menjadi hanya bisa ditulis saja oleh pemiliknya dengan command : **chmod 200 test.c**
7. Baca isi file **test.c** dengan **cat** atau **pico** atau **gedit**,
Apakah bisa? Mengapa?
8. Kembalikan hak akses file **test.c** seperti semula seperti point 3 di atas,
Apa command nya?
9. Compile file **test.c** dengan command: **gcc test.c -o test**
10. Jalankan file tersebut dengan command: **./test**
11. Lihat hak akses file **test** dengan command : **ls -l test**
Jelaskan arti hak akses nya !
12. Ubah hak akses file **test** dengan command : **chmod 600 test**
13. Jalankan kembali program **test** dengan command : **./test**
(Apakah bisa ? Jelaskan !)
14. Ubah hak akses file dengan letter coding **u**(user), **g**(group), **o**(other), **a**(ll),
+(menambah izin) dan **-**(mencabut izin). Contoh sbb:
chmod ug+rw test.c
chmod g-w test.c
Cobalah untuk mengubah hak akses dengan letter coding ini.

Tujuan Praktikum 2.3

1. Mhs mengerti dasar-dasar sistem file pada linux
2. Mhs mengerti instruksi untuk mencari file (**find**)
3. Mhs mengerti pipa (**pipe**)
4. Mhs mengerti **meta** karakter (karakter khusus)
5. Mhs mengerti **shell**

Petunjuk Praktikum 2.3

1. Untuk mencari file bisa gunakan command:
find <direktori-awal-pencarian> -name <file yg dicari> -print
contoh:
find /home -name alamat.txt -print
2. Lihat jenis-jenis file dengan command **ls -l**, dan lihatlah tanda paling kiri
Informasi Tambahan:
d => directory

- => regular file (file biasa)
 - b => block special file
 - c => character special file
 - l => symbolic link
 - p => pipe special file (FIFO)
 - s => socket file
3. Buat sebuah file dengan command **cat**
cat > asli.txt <tekan enter>
ketik sembarang saja di sini
tekan CTRL+D
 4. Buat link ke file **asli.txt** dengan command:
ln -s asli.txt slink.txt
Lihat tipe dari **slink.txt** dengan command: **ls -l slink.txt**
 5. Lihat isi file dari **slink.txt** dengan command: **cat slink.txt**
 6. Edit file **asli.txt** kemudian lihat **slink.txt**
 7. Hapus file **asli.txt** dan lihat file **slink.txt**.
Apakah masih terlihat?
 8. Coba command **ls -al** pada home directory Anda
 9. Coba command **ls -al | more** dan lihat perbedaannya dengan point sebelumnya
 10. Ada beberapa karakter khusus (meta karakter) yaitu : * \$ + ? { | } (\)
Jika ingin menggunakan karakter tersebut, tambahkan \.
Cobalah membuat nama file sbb:
touch lima\$dua
touch lima+dua
touch lima\\dua
Kemudian lihat nama-nama file tersebut dengan command: **ls**
 11. Coba beberapa command dengan karakter khusus di bawah ini:
ls *.txt
ls -l /bin/m?
ls -l /bin/m[a-z]
ls -l /bin/ma*
ls -l /bin/[m]*
ls -l /bin/m[a]*
ls -l /bin/m[!a]*
ls -l /bin/m[â]*
 12. Untuk melihat jenis-jenis shell yang ada pada komputer Anda
silahkan dilihat dengan command: **cat /etc/shells**
 13. Membuat alias (nama lain). Buatlah command dengan
nama "lihat_file" yang disingkat dengan **lf**.
Coba command **lf** apa sudah ada ?
 14. Buat alias dengan command : **alias lf=ls**.
Kemudian coba command **lf** apakah outputnya sama dengan **ls**?
 15. Buat alias dengan command : **alias lf='ls -al'**.
Kemudian coba command **lf** lagi dan perhatikan perubahan outputnya.
 16. **Global Regular Expression Parse** atau **grep** adalah perintah untuk mencari
file yang mengandung teks dengan kriteria yang telah Anda tentukan.
Format perintah:
grep <teks> <file> => Contoh: grep "this" test.txt
grep "this" test* => lihat semua file dengan nama awal test
 17. **gzip** adalah software kompresi zip versi GNU, fungsinya untuk
mengompresi sebuah file. Sintaksnya sangat sederhana:
gzip <namafile> => hasilnya adalah namafile.gz
gunzip namafile.gz => uncompress

18. Menyimpan dan mengekstrak file dari media seperti hard disk atau flash disk.

File arsip tersebut sering disebut sebagai file **tar**.

Sintaknya sebagai berikut:

tar <aksi> <option> <file atau direktori>

Contoh:

tar -czvf namaFile.tar.gz <nama-direktori>

tar -czvf namaFile.tar.gz <file yang dikompres dipisahkan dengan spasi>

tar -xzvf namaFile.tar.gz => untuk ekstrak file .tar.gz

tar -tzvf namaFile.tar.gz => untuk melihat file di dalam .tar.gz

19. Compress file menggunakan **zip** dan ekstrak dengan menggunakan **unzip**

Commandnya adalah:

zip <namaZip> <File yang akan dizip> => hasilnya adalah namaZip.zip

unzip <namaZip.zip> => untuk ekstrak file .zip

Tugas Praktikum 2

Sebelum keluar ruangan simpanlah history dari command yang telah anda buat ke text file.

Nama file **history_NIM.txt** dengan command:

history > history_NIM.txt

Compress dengan ekstensi **tar.gz** file **history_NIM.txt** dan **jawab_NIM.txt**

dengan nama baru **NIM_Nama.tar.gz**

Submit file tersebut ke asisten lab / dosen.

(apabila tidak ada koneksi internet, copy file tersebut via flashdisk dan kumpulkan)

Tujuan Praktikum 3

1. Mahasiswa mengerti dasar-dasar proses pada Linux.
2. Mahasiswa mengerti PID.
3. Mahasiswa mengerti parent, child, zombie dan orphan process.
4. Mahasiswa mengerti cara membuat proses dengan system call fork.
5. Mahasiswa mengerti program multithread dengan menggunakan POSIX library pthread.

Petunjuk Praktikum 3

1. Silahkan gunakan console 1 dengan menekan **CTRL + ALT + F1** dan login sebagai **student**, kemudian console 2 dengan **CTRL + ALT + F2** dan login sebagai user Anda sendiri.
2. Command: **w**
Digunakan untuk melihat informasi proses pada komputer Anda, atau perintah **w** dapat digunakan untuk menampilkan siapa saja yang sedang login dan proses apa saja yang sedang dilakukan oleh setiap pemakai.
Syntax: **w** atau **w <nama-user>** untuk user tertentu saja.
3. Command: **ps**
Perintah **ps** digunakan untuk menampilkan informasi status proses yang sedang dijalankan oleh pemakai.
Syntax: **ps <option> [-u UID]**
Options:
 - a all, semua user yang login.
 - e everything, seluruh sistem.
 - f full, ditampilkan dalam format yang lengkap.
 - l long, ditampilkan dalam format yang lengkap.
 - u UID, menampilkan info khusus untuk user tertentu.Contoh: **ps**, **ps a**, **ps al**, **ps aux**, **ps e**, dll.
4. Command: **pstree**
Digunakan untuk menampilkan daftar proses daemon sistem dalam bentuk tree (pohon proses). Setiap daemon yang aktif dan melayani proses dari pemakai terlihat memiliki anak proses.
5. Command: **top**
Digunakan untuk menampilkan informasi proses yang ada dalam sistem yang sedang berjalan. Aplikasi ini dapat dikatakan sebagai command **ps** yang interaktif.
6. Command: **kill**
Perintah ini digunakan untuk mengirim sinyal kepada suatu proses. Untuk mematikan suatu proses dapat menggunakan suatu sinyal nomor 9 (**SIGKILL**) atau 15 (**SIGTERM**).
7. Proses Background
Tanda **&** digunakan untuk memberitahukan kepada sistem agar proses dari perintah yang diberikan dapat dijalankan secara background.
Syntax: **<nama program> &**
Setiap perintah yang dijalankan secara background akan memiliki nomor job [x] dan PID.
Contoh: **[1] 22751**

Cobalah pada program **test.c** yang dibuat di bawah.

8. Command: **sleep**

sleep merupakan perintah untuk membuat suatu proses tidur (block).

Syntax: **sleep <detik>**

9. Buatlah program di bawah ini, beri nama **test.c**

```
int main(){
    int i;
    for (i=0; i<30; i++) sleep(1);
    return 0;
}
```

Compile dengan command: **gcc -o test test.c**

Jalankan di foreground dengan command: **./test**

Jika sleepnya terlalu lama hentikan dengan menekan **CTRL + C** atau kill dengan command: **kill -9 <pid-proses>**

Untuk melihat pid-proses gunakan command: **ps aux**

10. Jalankan program **test** secara background dengan command:

./test &

11. Command: **jobs**

Instruksi untuk mengetahui ada berapa proses background yang dimiliki oleh user.

Contoh: **sleep 5s &** kemudian ketik **jobs**

12. Command: **bg**

Suatu proses yang ditunda prosesnya dengan sinyal **CTRL + Z**, dapat diaktifkan lagi dengan command ini.

Contoh:

Jalankan program **test** di atas secara foreground, kemudian tekan **CTRL + Z**.

Kemudian setelah beberapa detik ketik **bf**

13. Command: **fg**

Merupakan perintah yang digunakan untuk mengalihkan suatu proses background menjadi foreground.

Contoh:

Jalankan program **test** di atas secara background dengan **./test &** kemudian setelah beberapa detik ketik command **fg**

14. Buatlah program di bawah ini dan beri nama **prak3-1.c**

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
```

```
void parentProcess();
void childProcess1();
void childProcess2();
```

```
int main() {
    pid_t child1;
    int status;
    child1 = fork();
    if(child1 < 0) {
        perror("fork1");
        exit(1);
    }
    //child process-1
    if(child1 == 0) childProcess1();
    //parent process
    if(child1 > 0) parentProcess();
    waitpid(child1, &status, 0);
    printf("Child Process-1 sudah selesai.....\n");
    return 0;
}

void parentProcess() {
    pid_t child2;
    int status;
    child2 = fork();
    if(child2 < 0) {
        perror("fork2");
        exit(1);
    }
    //child process-2
    if(child2 == 0) childProcess2();
    //parent process
    if(child2 > 0) {
        int i;
        for(i=0; i<10; i++) {
            printf("%d. Parent PID = %d\n", i++, getpid());
            printf("%d. Parent PPID = %d\n", i, getppid());
            sleep(2);
        }
        waitpid(child2, &status, 0);
        printf("Child Process-2 sudah selesai.....\n");
    }
}

void childProcess1() {
    int i;
    for(i=0; i<10; i++) {
        printf("\t%d. Child1 PID = %d\n", i++, getpid());
        printf("\t%d. Child1 PPID = %d\n", i, getppid());
        sleep(2);
    }
}
```

```
    }
    exit(0);
}
void childProcess2() {
    int i;
    for(i=0; i<10; i++) {
        printf("\t%d. Child2 PID = %d\n", i++, getpid());
        printf("\t%d. Child2 PPID = %d\n", i, getppid());
        sleep(2);
    }
    exit(0);
}
```

Kompilasi program tersebut dan jalankan.

Perhatikan outputnya, kemudian ubah waktu delay / sleep agar childProcess1 selesai terlebih dahulu.

Ubah waktu delay / sleep agar childProcess2 selesai terlebih dahulu kemudian kompilasi ulang dan jalankan.

Ubah waktu delay / sleep agar parentProcess selesai terlebih dahulu kemudian kompilasi ulang dan jalankan.

15. Buatlah program di bawah ini dan beri nama **prak3-2.c**

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

void thread1();
void thread2();
int g1=0;

int main() {
    int i;
    pthread_t thrd1, thrd2;
    int ret;
    ret=pthread_create(&thrd1, NULL, (void*)thread1, NULL);
    if(ret) {
        perror("pthread_create: thread1");
        exit(EXIT_FAILURE);
    }
    ret=pthread_create(&thrd2, NULL, (void*)thread2, NULL);
    if(ret) {
        perror("pthread_create: thread2");
        exit(EXIT_FAILURE);
    }
    pthread_join(thrd2, NULL);
    pthread_join(thrd1, NULL);
    exit(EXIT_SUCCESS);
}

void thread1() {
    while(g1 < 5) {
```

```
        printf("thread1: g1=%d\n", g1);
        g1++;
        sleep(1);
    }
}

void thread2() {
    while(g1 < 5) {
        printf("thread2: g1=%d\n", g1);
        g1++;
        sleep(1);
    }
}
```

Kompilasi program dengan perintah **gcc -o prak3-2 prak3-2.c -lpthread**

dan jalankan program tersebut! Perhatikan output program.

Coba hilangkan **sleep(1)** pada thread2 (i), thread1 (ii), kemudian keduanya (iii). Perhatikan output pada (i), (ii), dan (iii).

Tugas Praktikum 3

Jawaban ditulis dalam sebuah text file dengan nama **jawab3.txt**

1. Pada program **prak3-1.c** gambarkan hierarki proses! Apakah timbul Zombie proses? Apakah timbul Orphan proses? Jelaskan jika ada!
2. Bagaimana membuat program terbebas dari timbulnya baik Zombie maupun Orphan proses?
3. Pada program **prak3-2.c**, jelaskan struktur kedua buah thread terhadap proses! Apa yang terjadi ketika system call **sleep(1)** dihilangkan? Jelaskan kesimpulannya!
4. Sebutkan dan jelaskan 3 perbedaan implementasi program antara **prak3-1.c** dan **prak3-2.c**!

Tujuan Praktikum 4

1. Mahasiswa mampu menggunakan named pipe dalam komunikasi antar proses.
2. Mahasiswa mengerti cara membuat, menggunakan, dan menghapus shared memory.
3. Mahasiswa mampu menggunakan shared memory pada multi-proses dan mengenali adanya kondisi race condition.
4. Mahasiswa mampu menggunakan message passing.

Petunjuk Praktikum 4.1

1. Buat program seperti di bawah ini dan beri nama **prak4-1a.c**, kemudian compile dengan **gcc -o prak4-1a prak4-1a.c**
2. Buat FIFO dengan nama **fifo1** melalui terminal dengan command: **mkfifo -m 660 fifo1**

FIFO juga sering disebut dengan **named pipe** dan berada secara permanen pada sistem file, lihat file **fifo1** dengan command: **ls -al**

Kode Program prak4-1a.c

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <limits.h>

int main(){
    int fd; //descriptor untuk FIFO
    int len;
    char buf[PIPE_BUF];
    if((fd=open("fifo1",O_RDONLY))<0){
        perror("open");
        exit(EXIT_FAILURE);
    }

    while((len=read(fd,buf,PIPE_BUF-1))>0) printf("Read: %s", buf);
    close(fd);
    exit(EXIT_SUCCESS);
}
```

3. Buat program seperti di bawah ini dan beri nama **prak4-1b.c**, kemudian compile dengan **gcc -o prak4-1b prak4-1b.c**
File **prak4-1a**, **prak4-1b**, dan **fifo1** harus berada pada direktori yang sama.
4. Jalankan program **prak4-1a** pada terminal kemudian jalankan juga program **prak4-1b** pada terminal lainnya.
Perhatikan output pada terminal program **prak4-1a**.
5. Hapus instruksi **close(fd)**; dari keduanya, kemudian re-compile

dan jalankan lagi kedua program tersebut, kemudian berikan kesimpulan dari percobaan di atas !

Jawaban ditulis dengan nama file **kesimpulan4.txt**

Kode Program prak4-1b.c

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <limits.h>
#include <time.h>

int main(){
    int fd;
    int len;
    char buf[PIPE_BUF];
    time_t tp;
    if((fd=open("fifo1",O_WRONLY))<0){
        perror("open");
        exit(EXIT_FAILURE);
    }
    while(1){
        time(&tp); //ambil waktu
        len=sprintf(buf,"Dikirim oleh PID=%d Jam:%s",getpid(),ctime(&tp));
        if((write(fd,buf,len+1))<0){
            perror("write");
            close(fd);
            exit(EXIT_FAILURE);
        }
        sleep(3);
    }
    close(fd);
    exit(EXIT_SUCCESS);
}
```

Petunjuk Praktikum 4.2

1. Lihat shared memory yang sudah ada pada komputer Anda dengan perintah:
ipcs -m
2. Buat program seperti di bawah ini dan beri nama **prak4-2a.c**
3. Compile program dengan perintah **gcc -o prak4-2a prak4-2a.c** dan jalankan program dengan perintah: **./prak4-2a "Praktikum shared memory"**
4. Buka **terminal** yang lain dan jalankan program lagi dengan command **./prak4-2a**
Perhatikan output dari program, bila tidak mengerti, tanyakan kepada asisten lab

5. Lihat shared memory pada komputer Anda lagi dengan command: **ipcs -m**
Apakah **shmid** (Shared memory ID) bertambah?
6. Jalankan program kembali dengan perintah:
./prak4-2a "Selamat mencoba IPC dengan Shared Memory"
7. Buka **terminal** yang lain dan jalankan program lagi
dengan command **./prak4-2a**
Perhatikan output dari program, apabila tidak mengerti tanyakan pada asisten.
Bandingkan program1 dan program2, mengapa pada program1 ada output yang ditampilkan pada layar? tulis jawaban dengan nama **kesimpulan4.txt**

Listing Program prak4-2a.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>

#define SHM_SIZE 1024 //1KByte Shared Memory

int main(int argc, char* argv[]){
    key_t key;
    int shmid, mode;
    char *data;

    if(argc>2){
        fprintf(stderr,"usage: ./prak4-2 <data string>\n");
        exit(1);
    }

    //membuat key
    key = ftok("shmdemo.c",'R');

    //connect ke segment memory atau create segment memory
    shmid = shmget(key,SHM_SIZE,0644 | IPC_CREAT);

    //attach ke segment untuk mengarah pada pointer
    data = (char*)shmat(shmid,(void*)0,0);

    //read atau modifikasi segment, berdasarkan command yang ada.
    if(argc == 2){
        printf("write: \"%s\"\n",argv[1]);
        strncpy(data,argv[1],SHM_SIZE);
    }
    else{
        printf("read: \"%s\"\n",data);
    }

    //detach dari segment
```

```
    shmdt(data);  
    return 0;  
}
```

8. Lihat kembali shared memory pada komputer Anda: **ipcs -m**
9. Buat program seperti di bawah dan beri nama **prak4-2b.c**
10. Compile program dengan perintah **gcc -o ShmRemove prak4-2b.c**
11. Hilangkan shared memory segment atau **shmid** yang dibuat pada program **prak4-2a.c** di atas dengan program **prak4-2b.c** dengan command:
./ShmRemove <shmid>
12. Lihat shared memory segment kembali, apakah sudah hilang?

Listing Program prak4-2b.c

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <sys/types.h>  
#include <sys/ipc.h>  
#include <sys/shm.h>  
  
int main(int argc, char *argv[]) {  
    int shmid;  
    if(argc != 2) {  
        fprintf(stderr, "usage: %s <shmid>\n", argv[0]);  
        exit(1);  
    }  
    shmid=atoi(argv[1]);  
    shmctl(shmid, IPC_RMID, (struct shm_id *) 0);  
    return 0;  
}
```

Petunjuk Praktikum 4.3

1. Buat program seperti di bawah dan beri nama **prak4-3a.c**
Program di bawah adalah program dengan dua proses (child dan parent) berkomunikasi dengan shared memory
2. Compile program dan jalankan, kemudian ubah nilai pada **sleep(1)** yang ada pada child() dan parent(), perhatikan output, kemudian beri kesimpulan
(tulis pada **kesimpulan4.txt**)

Listing Program prak4-3a.c

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <sys/types.h>  
#include <sys/ipc.h>  
#include <sys/shm.h>  
#define SHMKEY ((key_t) 5678)  
#define SHM_SIZE 1024  
  
void child();  
void parent();  
  
int main() {  
    pid_t pid;  
    int status;
```

```
switch(pid= fork()) {
    case -1: perror("fork"); exit(1);
    case 0: child(); exit(EXIT_SUCCESS);
    default: parent(); waitpid(pid, &status);
}
exit(EXIT_SUCCESS);
}

void child() {
    int i, shmid;
    char *data;

    shmid=shmget(SHMKEY, SHM_SIZE, 0644 | IPC_CREAT);
    data = (char*) shmat(shmid, (void*)0, 0);

    for(i=0; i<5; i++) {
        sprintf(data, "i=%d\n", i);
        sleep(1);
    }

    shmdt(data);
}

void parent() {
    int i, shmid;
    char *data;

    shmid=shmget(SHMKEY, SHM_SIZE, 0644 | IPC_CREAT);
    data = (char*) shmat(shmid, (void*)0, 0);

    for(i=0; i<5; i++) {
        printf("\tParent: %s", data);
        sleep(1);
    }

    shmdt(data);
}
```

3. Program dibawah ini menunjukkan permasalahan **race condition** saat menggunakan shared memory.
4. Buat program seperti di bawah ini dan beri nama **prak4-3b.c**
5. Compile program dan jalankan berkali-kali. Apakah hasilnya tidak selalu sama?

Listing program prak4-3b.c

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/types.h>
#include <sys/shm.h>
#include <sys/ipc.h>

int main()
{
    int child, shmid, key=1000, *ptrData;
    child=fork();
    shmid=shmget(key, sizeof(int), 0644 | IPC_CREAT);
    ptrData=(int*) shmat(shmid, (void*)0, 0);
    *ptrData=0;
```

```
if(child==0)
{
    int i=0;
    for(i;i<15;i++)
    {
        *ptrData=*ptrData+1;
        printf("i=%d child's ptrData=%d\n",i,*ptrData);
    }
}
else if(child>0)
{
    int i=0;
    for(i;i<15;i++)
    {
        *ptrData=*ptrData+1;
        printf("i=%d parent's ptrData=%d\n",i,*ptrData);
    }
    int status;
    waitpid(child,&status,0);
}
shmdt(ptrData);
shmctl(shmid,IPC_RMID,(struct shmid_ds *)0);
return 0;
}
```

Petunjuk Praktikum 4.4

1. Buat program seperti di bawah dan beri nama **msgrecv.c** dan **msgsend.c**
2. Compile program dan jalankan pada 2 terminal yang terpisah
3. Pelajari cara kerja program ini agar bisa mengerjakan soal praktikum selanjutnya

Listing Program msgrecv.c

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

char ss[100];

int main() {
    int msqid;
    int i;
    key_t key = 4444;
    if( (msqid = msgget(key, 0644 | IPC_CREAT)) == -1) {
        perror("msgget");
        exit(1);
    }
    printf("Ready to receive messages...\n");
    for(;;) {
        if(msgrcv(msqid, (char*)ss, 100, 0, 0) == -1) {
            perror("msgrcv");
            exit(1);
        }
        for(i=0; i<100; i++)
            if(ss[i] == '\n')
                ss[i] = 0;
    }
}
```

```
        printf("\"\"%s\"\\n\", ss);
    }
    return 0;
}
```

Listing Program msgsend.c

```
#include <stdio.h>
#include <stdlib.h>
#include <errno.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>

char ss[100];

int main() {
    int msqid;
    key_t key = 4444;
    if( (msqid = msgget(key, 0644 | IPC_CREAT)) == -1) {
        perror("msgget");
        exit(1);
    }
    printf("Enter lines of text, ^D to quit:\\n");
    while(fgets(ss,100,stdin) && !feof(stdin)) {
        if(msgsnd(msqid, (char*)ss, 100, 0) == -1)
            perror("msgsnd");
    }
    if(msgctl(msqid, IPC_RMID, NULL) == -1) {
        perror("msgctl");
        exit(1);
    }
    return 0;
}
```

Tujuan Praktikum 5

1. Mahasiswa memahami konsep komunikasi antar proses beserta permasalahan turunannya
2. Mahasiswa memahami dan mampu mengimplementasi *mutex* dan *semaphore* untuk menangani permasalahan *race condition*

Materi Pembelajaran

Inter-Process Communication

Dalam hal *Inter-Process Communication* (IPC) terdapat tiga isu utama yang terkait dengan sistem operasi:

1. Bagaimana cara proses dapat saling berkomunikasi satu dengan lainnya (bertukar informasi)?
2. Bagaimana cara proses tidak saling mendahului yang mengakibatkan kesalahan output (*race condition*)?
3. Bagaimana cara menciptakan *proper sequencing* terhadap proses-proses tersebut?

Pada sesi praktikum ini, ketiga poin yang telah dijabarkan sebelumnya akan dijawab dengan menggunakan mekanisme *shared memory* dan *semaphore*.

Shared Memory

Shared memory adalah salah satu mekanisme IPC asli bawaan UNIX. Intinya adalah terdapat dua atau lebih proses yang saling berbagi segmen memori bersama, yang mereka dapat baca dan tulis untuk saling berkomunikasi satu sama lain. *Shared memory* memiliki definisi seperti itu, tidak lebih dan tidak kurang, hanya sepotong memori. Setelah posisi / alamat / lokasi segmen memori bersama tersebut diketahui oleh proses-proses, maka penggunaannya oleh proses dapat dilakukan menggunakan *system call* terkait, *system call* yang dimaksud ini penggunaannya sama contohnya seperti memanggil fungsi **malloc()** ketika *programmer* ingin mengalokasikan memori dinamis pada program.

Mekanisme *shared memory* adalah mekanisme tercepat dalam hal IPC. Untuk menggunakan *shared memory*, berikut ini adalah *library* yang harus disertakan:

```
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
```

Syntax yang dapat digunakan dalam *shared memory* adalah sebagai berikut.

Syntax	Fungsi
int shmid;	Menampung ID <i>shared memory</i> yang akan digunakan.
struct SHM_SIZE{ char str[512]; int flag, readcount; }	Struct untuk menampung <i>shared memory</i> di dalam program.
struct SHM_SIZE *var;	Membuat sebuah <i>pointer</i> bernama var dengan format SHM_SIZE yang menunjuk ke sebuah <i>shared memory</i> .
key_t key	Mendeclare sebuah <i>variable</i> key_t yang digunakan sebagai kunci untuk mendapatkan shmid.

shmget(key, sizeof(struct SHM_SIZE), IPC_MODE 0RWX);	Mendapatkan ID <i>shared memory</i> menggunakan key, dengan bentuk SHM_SIZE, dengan modifikasi IPC_MODE (Mode ada tabel di bawah). [Return Integer]	
	IPC_CREAT	Membuat <i>Segment Shared Memory</i> .
	IPC_EXCL	Memastikan IPC_CREAT untuk membetnuk <i>Segment</i> . (akan <i>return error</i> apabila IPC_CREAT tidak membuat <i>Segment</i>)
shmat(shmid,addr,flag);	Menghubungkan <i>pointer</i> ke <i>shared memory</i> yang ada dengan shmid, <i>address</i> addr(addr diisi NULL dan <i>flag</i> 0 karena untuk materi ini tidak digunakan).	

Semaphore

Semaphore merupakan mekanisme IPC yang definisi sederhananya adalah variabel dengan tipe data *integer*, dan digunakan sebagai kendali / penanda untuk mengakses *critical region* suatu program. *Semaphore* seringkali digunakan untuk menghindari masalah *race condition* yang mana sama seperti *mutex*. Untuk menggunakan *semaphore*, berikut ini adalah *library* yang harus disertakan:

```
#include <semaphore.h>
```

Berikut ini merupakan *syntax* yang dapat dipakai untuk menggunakan *semaphore*.

Syntax	Fungsi								
<code>sem_t *var;</code>	Menyatakan (<i>declare</i>) <i>semaphore</i> dengan nama <code>var</code> .								
<code>sem_open(SEM_NAME, MODE, 0RWX, INIT_VALUE);</code>	Membuka <i>semaphore</i> untuk digunakan dengan nama <code>SEM_NAME</code> , <code>MODE</code> yang ingin digunakan, <code>RWX</code> bit, dan nilai awal dari <i>semaphore</i> .								
	<table><tr><th>Mode</th><th>Fungsi</th></tr><tr><td><code>O_CREAT</code></td><td>Membuat <i>semaphore</i> baru jika tidak ada. (Tidak akan membuat jika sudah ada)</td></tr><tr><td><code>O_EXCL</code></td><td>Memastikan <code>O_CREAT</code> membuat <i>semaphore</i>. (akan <i>return error</i> apabila <code>O_CREAT</code> tidak membuat <i>semaphore</i>)</td></tr><tr><td><code>O_RDONLY</code></td><td>Mencari <i>semaphore</i> sesuai dengan <code>SEM_NAME</code>.</td></tr></table>	Mode	Fungsi	<code>O_CREAT</code>	Membuat <i>semaphore</i> baru jika tidak ada. (Tidak akan membuat jika sudah ada)	<code>O_EXCL</code>	Memastikan <code>O_CREAT</code> membuat <i>semaphore</i> . (akan <i>return error</i> apabila <code>O_CREAT</code> tidak membuat <i>semaphore</i>)	<code>O_RDONLY</code>	Mencari <i>semaphore</i> sesuai dengan <code>SEM_NAME</code> .
	Mode	Fungsi							
	<code>O_CREAT</code>	Membuat <i>semaphore</i> baru jika tidak ada. (Tidak akan membuat jika sudah ada)							
	<code>O_EXCL</code>	Memastikan <code>O_CREAT</code> membuat <i>semaphore</i> . (akan <i>return error</i> apabila <code>O_CREAT</code> tidak membuat <i>semaphore</i>)							
<code>O_RDONLY</code>	Mencari <i>semaphore</i> sesuai dengan <code>SEM_NAME</code> .								
<code>sem_wait(var);</code>	Menurunkan nilai <i>semaphore</i> sebanyak 1 poin (apabila <i>semaphore</i> ≤ 0 , maka proses akan diblok)								
<code>sem_post(var);</code>	Menaikkan nilai <i>semaphore</i> sebanyak 1 poin.								
<code>sem_unlink(SEM_NAME);</code>	Melepas <i>semaphore</i> <code>SEM_NAME</code> dari <i>Shared Memory</i> .								

<code>sem_close(var);</code>	Menutup variabel <i>semaphore</i> dengan nama <i>var</i> .
------------------------------	--

Inter-Process Communciation

Dalam komunikasi antar *thread*, terdapat *library* bawaan dari UNIX yang dapat digunakan yaitu POSIX Thread (*Pthread*). *Pthread* merupakan *library* untuk menciptakan dan mengelola *thread*. Program yang memanfaatkan kemampuan *multithreading* memungkinkan adanya peningkatan performa yang dikarenakan adanya lebih dari satu alur eksekusi pada program. Tantangan utama dalam hal ini adalah proses sinkronisasi antara tiap *thread* yang ada. Untuk menggunakan fungsi *Pthread*, berikut ini adalah *library* yang harus disertakan:

```
#include <pthread.h>
```

Kemudian berikut ini merupakan *syntax* yang dapat digunakan.

Syntax	Fungsi
<code>pthread_mutex_t kunci = PTHREAD_MUTEX_INITIALIZER</code>	Membuat <i>Mutex</i> dengan nama kunci.
<code>pthread_t var;</code>	Membuat variabel <i>thread</i> dengan nama <i>var</i> .
<code>pthread_create(&var, &attr, (*function), &argument);</code>	Menciptakan <i>thread</i> "var" dan atribut "attr", yang mana <i>thread</i> ini akan mengerjakan/menjalankan fungsi "function". (Nilai <i>attribute</i> dan <i>argument</i> dapat diisi NULL apabila tidak ingin digunakan).
<code>pthread_join(var, **value_ptr);</code>	Menghubungkan kembali <i>thread</i> "var" dengan alur eksekusi <i>thread</i> utama. Bila berhasil maka fungsi akan mengembalikan nilai 0. (Nilai dari <i>value_ptr</i> dapat diisi dengan NULL apabila tidak digunakan).
<code>pthread_mutex_lock(kunci);</code>	Menutup kunci <i>mutex</i> .
<code>pthread_mutex_unlock(kunci);</code>	Membuka kunci <i>mutex</i> .

Petunjuk Praktikum 5.1

Lengkapilah potongan kode di bawah ini dan simpan, kemudian coba jalankan **Prak5-1a.c** terlebih dahulu sebelum **Prak5-1b.c**. **Amati** dan **pahami** hasil eksekusi dari program kemudian jelaskan cara kerja kedua program ini dan simpan dengan nama *file* "jawab.txt".

Prak5-1a.c

```
#include <stdio.h>;
#include <semaphore.h>;
#include <fcntl.h>;

int main(){
    int i;
    sem_t *mutex;
    ...
    if(mutex == SEM_FAILED){
        ... //apabila semaphore gagal dibuat
    }
    for(i = 0; i < 10; i++){
        ...
    }
}
```



```
        printf("Intro to Semaphore...\n");  
    }  
    ...  
    return 0;  
}
```

Prak5-1b.c

```
#include <semaphore.h>  
#include <fcntl.h>  
#include <stdlib.h>  
#include <stdio.h>  
  
int main(){  
    int i;  
    ...  
    if(mutex == SEM_FAILED){  
        perror("sem_open()");  
        exit(1);  
    }  
    for(i = 0; i < 10; i++){  
        printf("Tekan enter: ");  
        getchar();  
        ...  
    }  
    ...  
    return 0;  
}
```

Petunjuk Praktikum 5.2

Perhatikan potongan kode **Prak5-2.c** di bawah ini. Lengkapilah dan coba dijalankan ulang berkali-kali, **amati** dan **pahami** hasil eksekusi program. Jawablah pertanyaan tambahan di bawah ini dan simpan kembali ke dalam *file* "jawab.txt".

1. Apakah program memberikan *output* yang konsisten? Jika tidak, mengapa?
2. Apakah program berhenti pada saat *runtime*? Jika iya, mengapa?
3. Bagaimana cara agar program tidak berhenti pada saat *runtime*?

Prak5-2.c

```
#include <unistd.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <pthread.h>  
#include <string.h>  
#include <errno.h>  
#include <semaphore.h>  
#include <fcntl.h>  
#include <signal.h>  
  
void *thread1();  
void *thread2();  
void sigint_handler(int sig);
```

```
int var1=0, var2=0;
sem_t *mut1, *mut2;

int main(){
    pthread_t thrd1, thrd2;
    signal(SIGINT, sigint_handler);
    mut1 = ...
    mut2 = ...
    ...
}

void *thread1(){
    int i;
    for(i=0; i<10; i++){
        ...
        var1++;
        ...
        printf("var1 + var2 = %d\n", var1+var2);
        ...
    }
}

void *thread2(){
    int i;
    for(i=0; i<10; i++){
        ...
        var2++;
        ...
        printf("var1 + var2 = %d\n", var1+var2);
        ...
    }
}

void sigint_handler(int sig){
    sem_close(mut1);
    sem_close(mut2);
    sem_unlink("PRAK5-2MUT1");
    sem_unlink("PRAK5-2MUT2");
    exit(0);
}
```

Petunjuk Praktikum 5.3

Berikutnya cobalah potongan kode di bawah ini kemudian jalankan, **amati** dan **pahami** hasil eksekusinya! Jelaskan bagaimana cara kerja **Prak5-3writer.c** dan **Prak5-3reader.c**? Simpanlah jawabannya ke dalam file "jawab.txt".

Prak5-3writer.c

```
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <semaphore.h>
#include <fcntl.h>
#include <stdlib.h>
#include <string.h>
```

```
#include <signal.h>
#include <unistd.h>

#define TRUE 1
#define FALSE 0

struct SHM_SIZE{
    char str[512];
    int flag, readcount;
};

sem_t *resourcemutex;
int shmid, EXIT;
void sigint_handler(int sig);
struct SHM_SIZE *ptr;

int main(){
    char str[512], ss[512];
    key_t key;
    key = 1053;
    EXIT = FALSE;

    if(signal(SIGINT, sigint_handler) == SIG_ERR){
        perror("SIGQUIT");
        exit(1);
    }

    resourcemutex = sem_open("PRAK5-3RESC", O_CREAT, 0644, 1);
    if(resourcemutex == SEM_FAILED){
        perror("unable to execute semaphore");
        sem_unlink("PRAK5-3RESC");
        exit(-1);
    }

    shmid = shmget(key, sizeof(struct SHM_SIZE), IPC_CREAT|0644);
    if(shmid < 0){
        perror("shmget error..");
        exit(-1);
    }
    ptr = shmat(shmid, NULL, 0);

    int i = 0;

    while(EXIT==FALSE){
        printf("Masukkan Kata/kalimat : ");
        scanf("%[^\n]", ss); getchar();
        sem_wait(resourcemutex);
        strcpy(ptr->str, ss);
        sem_post(resourcemutex);
        i++;
    }
    sem_close(resourcemutex);
```

```
    return 0;
}

void sigint_handler(int sig){
    EXIT=TRUE;
}
```

Prak5-3reader.c

```
#include <sys/ipc.h>
#include <sys/shm.h>
#include <stdio.h>
#include <semaphore.h>
#include <fcntl.h>
#include <stdlib.h>
#include <signal.h>

#define TRUE 1
#define FALSE 0

struct SHM_SIZE{
    char str[512];
    int flag, readcount;
};

int EXIT;

void sigint_handler(int sig);

int main(){
    int shmid;
    int i, j;
    char ss[512], str[512];
    key_t key;
    struct SHM_SIZE *ptr;
    sem_t *readermutex, *resourcemutex;
    EXIT=FALSE;
    key=1053;
    if(signal(SIGINT, sigint_handler) == SIG_ERR){
        perror("SIGQUIT");
        exit(1);
    }

    readermutex = sem_open("PRAK5-3READ", O_CREAT, 0644, 1);
    if(readermutex == SEM_FAILED){
        perror("unable to create semaphore");
        sem_unlink("PRAK5-3READ");
        exit(-1);
    }

    resourcemutex = sem_open("PRAK5-3RESC", O_CREAT, 0644, 1);
    if(resourcemutex == SEM_FAILED){
        perror("unable to create semaphore");
        sem_unlink("PRAK5-3RESC");
        exit(-1);
    }
}
```

```
    shmids = shmget(key, sizeof(struct SHM_SIZE), IPC_CREAT | 0644);  
    if(shmids<0){  
        perror("failure in shmget");  
        exit(-1);  
    }  
    ptr = shmat(shmids, NULL, 0);  
  
    while(EXIT==FALSE){  
        sem_wait(resourcemutex);  
        if(ptr->flag == 0){  
            usleep(1000);  
            continue;  
        }  
        else{  
            printf("Reading %s\n", ptr->str);  
            ptr->flag = 0;  
        }  
        sem_post(resourcemutex);  
    }  
    return 0;  
}  
  
void sigint_handler(int sig){  
    EXIT=TRUE;  
    sem_close(readermutex);  
    sem_close(resourcemutex);  
    sem_unlink("PRAK5-3READ");  
    sem_unlink("PRAK5-3RESC");  
    exit(0);  
}
```

Tugas Mandiri

Dalam komputasi, **Producer-Consumer problem** atau juga disebut sebagai masalah *buffer-bounded* adalah contoh klasik dari masalah sinkronisasi di IPC. Masalahnya menggambarkan dua buah proses yaitu produsen dan konsumen, yang memiliki **buffer** dengan ukuran tetap (**fix**) dengan tipe pengelolaan **queue**. Tugas dari produsen (**producer**) adalah menghasilkan data (apapun itu bukan inti permasalahannya), lalu memasukkannya ke dalam *buffer* tadi, dan kemudian mengulang lagi seterusnya. Di saat yang bersamaan, konsumen (**consumer**) akan mengambil data dari *buffer* sekaligus menghapusnya dari *buffer*, proses pengambilannya satu per satu.

Masalahnya adalah untuk memastikan bahwa produsen tidak akan mencoba menambahkan data ke dalam *buffer* apabila sudah **penuhi** dan juga konsumen tidak akan mencoba untuk mengambil serta menghapus data dari *buffer* apabila sudah **kosong**.

Berdasarkan penjabaran permasalahan di atas, maka buatlah program kreasi Anda sendiri yang menggambarkan / menyimulasikan permasalahan *producer-consumer*. Program harus dikumpulkan paling lambat 1 hari sebelum pertemuan minggu ke-6!

Tujuan Praktikum 6

1. Mahasiswa memahami algoritma CPU scheduling pada Scheduler
2. Mahasiswa mampu mengimplementasi algoritma CPU scheduling

Petunjuk Praktikum 6.1

1. Buat program dengan nama **fcfs.c** program tersebut menerima input sebuah integer **N**, dimana **N** merupakan jumlah task yang ada. Setelah itu, program menerima input sebanyak **N** baris integer, dimana setiap baris terdiri atas 4 buah integer, yakni **I**, **arr**, **cpu**, dan **io**. Penjelasan dari keempat integer tersebut:

- **I** merupakan nama dari task yang datang.
- **arr** merupakan waktu task tersebut tiba.
- **cpu** merupakan waktu lamanya task tersebut menggunakan resource CPU.
- **io** merupakan waktu lamanya task tersebut menggunakan resource IO

Setelah input selesai dimasukkan, program akan melakukan scheduling terhadap semua task tersebut, dengan menggunakan algoritma scheduling **FCFS (First Come First Served)**. Kemudian program akan menampilkan **total waiting time**, **response time**, dan **turnaround time** dari setiap task. Terakhir, program akan menampilkan rata-rata dari **waiting time**, **response time**, dan **turnaround time**. Struktur data yang digunakan **bebas**, yang mana menurut Anda yang dapat digunakan.

Batasan dari input yang dimasukkan (tidak perlu dibatasi dengan if/switch, hanya input yang digunakan untuk menguji program yang anda buat)

```
3 ≤ N ≤ 1000
0 ≤ I ≤ 1000
0 ≤ arr ≤ 1000000
0 ≤ CPU ≤ 1000
0 ≤ IO ≤ 1000
```

Contoh Input

```
3
0 0 14 10
3 0 3 0
10 0 0 3
```

Hasil Output :

```
0 0 0 24
3 24 24 27
10 27 27 30
average = 17.00 17.00 27.00
```

Penjelasan Input

Angka 3 di baris pertama menandakan ada 3 buah task. Task pertama memiliki nama 0, task tersebut tiba pada waktu 0, task tersebut menggunakan resource CPU selama 14 satuan waktu, dan resource IO selama 10 satuan waktu. Task kedua memiliki nama 1, task tersebut tiba pada waktu 0, task tersebut menggunakan resource CPU selama 3 satuan waktu, dan tidak

menggunakan resource IO. Task ketiga memiliki nama 2, task tersebut tiba pada waktu 0, task tersebut tidak menggunakan resource CPU, dan menggunakan resource IO selama 3 satuan waktu.

CHALLENGE

Perbaikilah program **fcfs.c** agar dapat menghasilkan **average waiting time**, **average response time**, dan **average turnaround time terkecil!** Pada contoh input di atas, apabila program telah diperbaiki, output yang dihasilkan adalah

0 6 6 30

3 0 0 3

10 3 3 6

average = 3.00 3.00 13.00

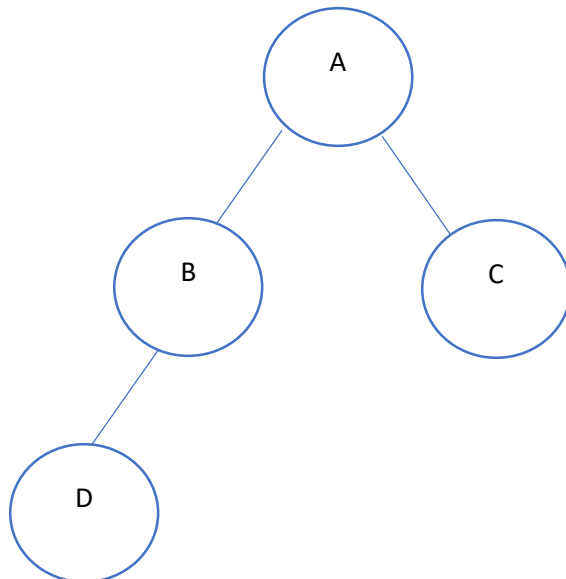
Gunakan seluruh sumber pustaka yang dapat kalian akses. Apabila tidak selesai, kerjakan di rumah dan kumpulkan pada minggu berikutnya!

Tujuan Praktikum 7

1. Quiz Pra-UTS

Soal Quiz

1.



Buatlah hirarki seperti di atas. B dan C akan mencetak PPID dan PID mereka masing-masing sebanyak 10 kali. Sedangkan A harus mencetak PID -nya sendiri serta PID dan PPID dari D sebanyak 10 kali juga. Gunakanlah pipe antara A dan D, agar D dapat mengirimkan PID nya ke A.

2. Anto, Brian, Caca, dan Desi adalah satu keluarga dan mereka saling bersaudara. Mereka berempat adalah anak yang rajin dan baik hati sehingga mereka ingin membantu orang tua mereka di rumah. Untuk membantu orang tua hanya diperlukan 1 anak saja per hari, sehingga perlu dibuat giliran/jadwal bagi mereka. Karena mereka juga anak yang inisiatif maka mereka mengambil giliran mereka sendiri masing-masing dengan ketentuan:
- Setiap anak harus membantu orang tua **tepat** sebanyak 5 kali
 - Urutan anak yang membantu setiap harinya bebas, namun 1 hari **hanya** 1 anak
 - Anak yang sudah selesai membantu sebanyak 5 kali akan beristirahat hingga semua saudaranya juga selesai.
 - Setelah semuanya selesai, orang tua akan memberikan hadiah kepada anak-anak tercintanya.

Contoh Output:

Anton: saya hari ke-1

Brian: saya hari ke-2

Caca: saya hari ke-3

Desi: saya hari ke-4

Anton: saya hari ke-5

Brian: saya hari ke-6

...

Caca: saya hari ke-20

Orang tua: ini hadiah buat kalian karena telah membantu

Praktikum 8

Memory management

Pokok Bahasan

Penerapan teknis memory management menggunakan bahasa pemrograman C



A. Tujuan Praktikum

1. Mahasiswa mampu menerapkan memory management yang baik menggunakan bahasa pemrograman C.
2. Mahasiswa mampu memahami dan mengatasi permasalahan *memory management* yang terjadi.
3. Mahasiswa mampu menggunakan alat bantu Valgrind dalam menangani permasalahan tersebut.

B. Petunjuk Praktikum

1. Jawaban pertanyaan yang berupa tulisan diketik dalam format .txt.
2. Simpan seluruh hasil jawaban Anda dalam folder yang disediakan di atas.
3. Pengumpulan terakhir dapat dilakukan paling lambat sebelum hari berikutnya.
4. Periksa kembali jawaban Anda, bila telah lengkap zip dengan format:
NIM_NAMA_OS-(Kode Kelas)_08.zip yang berisikan *text file* atas jawaban untuk Tugas Pendahuluan dan Tugas Praktikum.

C. Materi

Memory Management merupakan salah satu konsep penting dalam pemrograman dan juga Sistem Operasi pada umumnya. Programmer idealnya mampu menerapkan manajemen memori yang baik dalam membuat program/aplikasi. Salah satu contohnya adalah penggunaan memory yang seperlunya dan tidak melebihi kapasitas alokasi yang disediakan. Begitupun dengan proteksi memori yaitu sebuah program tidak boleh melakukan akses terhadap area memori yang tidak dialokasikan bagi program itu sendiri. Berikut ini adalah beberapa permasalahan umum dalam pengelolaan memori.

- *Premature frees and dangling pointers*
- *Memory leak*
- *External fragmentation*
- *Double frees*
- *Uninitialized variables*

Praktikum ini akan membahas beberapa permasalahan umum dalam manajemen memori menggunakan bahasa pemrograman C. Penggunaan Valgrind sebagai sebuah tool

untuk membantu dalam menemukan kesalahan manajemen memori pada program juga dibahas dalam praktikum ini.

D. Tugas Pendahuluan

1. Tool Valgrind adalah salah satu tool yang dapat Anda gunakan dalam menangani permasalahan manajemen memori. Selain tool ini sebutkan satu tool serupa lainnya! Deskripsikan dengan singkat tool lain tersebut!
2. Sebutkan sebuah keterbatasan dari tool Valgrind dalam mengatasi permasalahan manajemen memori!
3. Install Valgrind pada *virtual machine* Anda dengan menggunakan *command*: “**sudo apt-get install valgrind**”.

E. Tutorial

Buatlah program seperti di bawah ini dan simpan dengan nama latihan-8.c, pahami setiap instruksi dan maksud dari program berikut ini.

```
1 // NAMA
2 // NIM
3
4 #include<stdlib.h>
5 #include<stdio.h>
6 #include<string.h>
7
8 int* increment(){
9     int *x ;
10    x = (int *) malloc(sizeof(int));
11    int i;
12    for(i=0;i<10;i++){
13        x[0] +=1;
14        x[1] +=2;
15    }
16    return x;
17 }
18
19 char* tukar(){
20     char *a, *b, *c;
21     a = (char *) malloc(sizeof(char) * 50);
22     b = (char *) malloc(sizeof(char) * 50);
23     c = (char *) malloc(sizeof(char) * 50);
24
25     strcpy(a,"string a");
26     strcpy(b,"string b");
27
28     printf("\nBefore :\n");
29     printf("a = %s\n",a);
30     printf("b = %s\n",b);
31
32     c = b;
33     b = a;
34     a = c;
35
36     printf("\nAfter :\n");
37     printf("a = %s\n",a);
```

```
38     printf("b = %s\n",b);
39
40     free(a);
41     free(b);
42     free(c);
43
44     return c;
45 }
46
47 int main(){
48     int* x;
49     char *y;
50     x = increment();
51     printf("x0 = %d\n",x[0]);
52     printf("x1 = %d\n",x[1]);
53     y = tukar();
54     printf("%s\n",y);
55     getchar();
56     return 0;
57 }
```

Di sini Anda *compile* program di atas menggunakan *command*: “gcc -g -o latihan-8 latihan-8.c”. Selanjutnya, jalankan valgrind dengan menggunakan *command*: “valgrind --leak-check=yes ./latihan-8”.

F. Tugas Praktikum

1. Jelaskan dengan singkat maksud/tujuan dari program “latihan-8.c” di atas!
2. Jelaskan apa saja permasalahan manajemen memori yang terdapat pada program “latihan-8.c” di atas!
3. Jelaskan apa saja penyebab yang menimbulkan permasalahan-permasalahan tersebut!
4. Berdasarkan analisa Anda sebelumnya, maka perbaikilah program “latihan-8.c”!

Praktikum 09

Virtual Memory Address Translation

Pokok Bahasan

Implementasi tanslasi virtual address ke physical address dalam paging



A. Tujuan Praktikum

1. Mahasiswa memahami cara kerja *virtual memory* khususnya *paging* di sistem operasi.
2. Mahasiswa memahami proses penerjemahan alamat dari *virtual address* menjadi *physical address*.
3. Mahasiswa mampu mengimplementasikan cara kerja proses translasi dasar yang dilakukan oleh *Memory Management Unit (MMU)* dengan bantuan *page table* menggunakan bahasa pemrograman C.

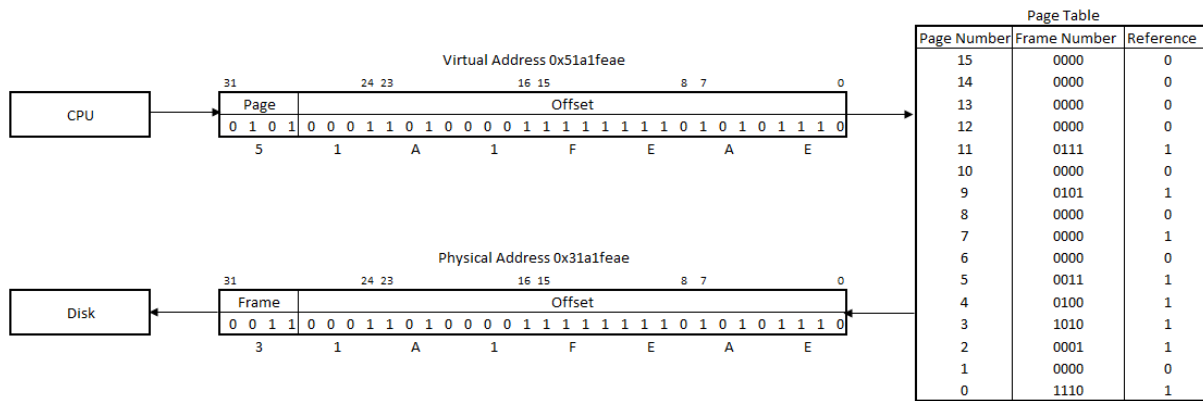
B. Petunjuk Praktikum

1. Jawaban pertanyaan yang berupa tulisan diketik dalam format .txt.
2. Jawaban pertanyaan yang berupa source code diketik dalam format .c.
3. Pengumpulan terakhir dapat dilakukan paling lambat sebelum praktikum selesai.
4. Periksa kembali jawaban Anda, bila telah lengkap compress dengan format: **NIM_NAMA_w09.tar.gz** yang berisikan *text file* dan *source code* atas jawaban untuk Tugas Pendahuluan dan Tugas Praktikum.

C. Materi

Virtual memory adalah teknik *memory management* yang memungkinkan pengguna untuk dapat menggunakan memori dengan kapasitas lebih besar dari kapasitas memori yang ada. Salah satu implementasi dari konsep *Virtual memory* adalah *paging*. *Paging* membagi proses menjadi *page* dan memori utama (RAM) menjadi *frame*. Dalam proses pemetaannya terdapat translasi alamat dari *virtual* ke *physical*. Hal ini dilakukan oleh komponen *hardware* yaitu MMU dengan memanfaatkan *page table* dan *Translation Lookaside Buffer (TLB)*.

Ketika program memerlukan suatu *page* yang tidak ada pada RAM, prosesor (CPU) dalam hal ini program yang sedang dieksekusi akan menghasilkan *virtual address* untuk ditranslasikan kemudian menjadi *physical address*. Berikut adalah ilustrasi sederhana proses translasi *virtual address* menjadi *physical address* (RAM) yang dikerjakan oleh MMU dengan memanfaatkan *page table* yang tersedia.



D. Tugas Pendahuluan

Jawablah pertanyaan berikut ke dalam file **jawab.txt**!

1. Jelaskan manfaat digunakannya *virtual memory* dibandingkan dengan *partitioning* dalam *memory management* di sistem operasi!
2. Sebutkan dan jelaskan dengan singkat teknik implementasi *virtual memory* selain *paging*!
3. Jelaskan dengan singkat apa yang akan dilakukan sistem operasi apabila seluruh *page frame* pada RAM sudah *overcommitted* pada saat terjadi *page fault*!

E. Tutorial

Tedja adalah seorang gamer sejati yang suka bermain sebuah game FPS berjudul Nonton Berlebih Online (NBO). Pada game tersebut, karena Tedja kalah terus-terusan, dia merasa kesal dan mencari cara untuk memenangkan game NBO dengan cara ilegal. Dengan menggunakan program MesinCurang (MC), Tedja berhasil menemukan alamat virtual yang menyimpan informasi mengenai jumlah peluru yang ada pada game NBO tersebut. Tedja ingin membuat jumlah pelurunya menjadi tidak terbatas, namun program MesinCurang (MC) tersebut tidak dapat mengubah alamat virtual jumlah peluru menjadi alamat fisik yang sebenarnya berada pada memori komputer Tedja. Anda sebagai sahabat Tedja diminta bantuannya untuk membuat sebuah program sederhana yang berfungsi mentranslasikan *virtual address* jumlah peluru menjadi *physical address*.

Buatlah program dengan nama **simple_mmu.c**, program ini akan mensimulasikan proses translasi *virtual address* menjadi *physical address* yang dilakukan oleh MMU. Input dari program secara terus-menerus adalah bilangan heksadesimal 8 digit, yang merupakan *virtual address* itu sendiri. Program kemudian mengolah alamat tersebut menjadi *physical address* dengan menggunakan *page table* yang telah disediakan. Output dari program adalah *physical address* dari setiap input yang diberikan. Apabila terjadi *page fault* maka program akan menampilkan pesan: "PAGE FAULT!". Hal ini dapat dilakukan dengan memeriksa *resident bit* pada *page table entry*.

Batasan dari program adalah input *virtual address* dan output *physical address* merupakan bilangan bulat positif 32-bit dengan 4 *most significant bits*-nya sebagai *page number* dan *frame number* masing-masing. Sisa bit adalah *offset* dari *address* tersebut.

Berikut ini adalah informasi *page table* yang akan digunakan untuk membuat program `simple_mmu.c` pada praktikum ini.

PAGE TABLE SIMPLE_MMU

Page Number	Frame Number	Resident Bit
15	X	0
14	1100	1
13	X	0
12	X	0
11	0111	1
10	X	0
9	0101	1
8	X	0
7	X	0
6	X	0
5	0011	1
4	0100	1
3	1010	1
2	0001	1
1	X	0
0	1110	1

Contoh Input

0x51A1FEAE

0xFFfFfFfF

0x00000000

Contoh Output

0011 0001 1010 0001 1111 1110 1010 1110 (0x31A1FEAE)

PAGE FAULT!

1110 0000 0000 0000 0000 0000 0000 0000 (0xE0000000)

Penjelasan Contoh Kasus

Dari input 0x51A1FEAE dijabarkan menjadi bentuk 32 bit yaitu: 0101 0001 1010 0001 1111 1110 1010 1110. Dengan menggunakan (4 *most significant bit*) sebagai *page number* (0101 yaitu 5), tentukan *frame number* dari alamat tersebut (atau apakah terjadi *page fault*). Setelah mendapatkan *frame number*, ubah susunan bit menjadi *physical address* menggunakan informasi *frame number*, yaitu: 0011 0001 1010 0001 1111 1110 1010 1110. Setelah mendapatkan *physical address* yang diinginkan, tampilkan alamat tersebut dalam bentuk biner dan heksadesimal dengan format seperti pada contoh output di atas.

Praktikum 10

Page Replacement Algorithm

Pokok Bahasan

Implementasi algoritma penggantian page dengan bahasa pemrograman C



A. Tujuan Praktikum

1. Mahasiswa memahami algoritma penggantian page pada memory management
2. Mahasiswa mampu mengimplementasikan algoritma pergantian page menggunakan bahasa pemrograman C

B. Petunjuk Praktikum

1. Jawaban pertanyaan yang berupa tulisan diketik dalam format .txt.
2. Simpan seluruh hasil jawaban Anda dalam folder yang disediakan di atas.
3. Pengumpulan terakhir dapat dilakukan paling lambat sebelum hari berikutnya.
4. Periksa kembali jawaban Anda, bila telah lengkap compress dengan format:
NIM_NAMA_OS-(Kode Kelas)_10.tar.gz yang berisikan *text file* atas jawaban untuk Tugas Pendahuluan dan Tugas Praktikum.

C. Materi

Pada *virtual memory systems*, implementasi paging umum digunakan. Salah satu isu dalam paging adalah efisiensi dan kecepatan pergantian page (saat terjadi *page fault*). Dalam hal ini, Sistem Operasi sebagai pengatur sumber daya memori sebuah komputer harus dapat melakukan pergantian page dengan cepat dan efisien, salah satu caranya adalah dengan menerapkan *page replacement algorithm* seperti contoh berikut ini.

- Optimal Page Replacement Algorithm
- Not Recently Used
- First In, First Out
- Second Chance
- Clock

Pada praktikum kali ini, pergantian page akan dilakukan dengan menggunakan algoritma *second chance*. Implementasi pergantian page ini dapat dilakukan dengan bantuan sebuah program sederhana untuk mensimulasikan proses pergantian page dengan menggunakan algoritma tersebut.

D. Tugas Pendahuluan

1. Pahami algoritma *second chance* dengan baik sebelum memulai sesi praktikum ini.
2. Jelaskan dengan singkat langkah-langkah atau aturan main dari algoritma *second chance*!

E. Tutorial

Buatlah program dengan nama **second_chance.c**, program ini mensimulasikan *page replacement* dengan menggunakan algoritma *second chance*. Program ini akan menerima input integer **PN**, yaitu *page number* dan dibatasi antara 1 sampai dengan 5 saja. Hal ini merepresentasikan *page number* yang dibutuhkan/diminta oleh prosesor (CPU).

Setelah input selesai dimasukkan, program akan mensimulasikan algoritma *second chance* sesuai dengan input. Kemudian program akan menampilkan jumlah **page fault** yang terjadi, dan menampilkan kondisi terakhir dari buffer, beserta dengan reference bit dari masing-masing page di dalam buffer.

Batasan dari program adalah ukuran *page table* yang hanya dapat menampung sebanyak 3 *page number* saja. Program ini akan terus berjalan sampai dimasukkan input **0**.

Contoh Input pertama kali

1

Contoh Output pertama kali

PF = 1

PN R-bit

1 0

X X

X X

Contoh Input kedua kali

2

Contoh Output kedua kali

PF = 2

PN R-bit

2 0

1 0

X X

Sehingga apabila dimasukkan input permintaan page number sebagai berikut secara sekuensial: 1 2 3 5 5 3 4 2 3 1 3. Maka akan diperoleh hasil akhir tampilan page table sebagai berikut.

PF = 7

PN R-bit

1 0

3 1

2 0

Praktikum 11

Disk Scheduling Algorithm

Pokok Bahasan

Pemahaman algoritma penjadwalan disk: SSTF dan SCAN



A. Tujuan Praktikum

1. Mahasiswa memahami algoritma yang digunakan untuk *disk scheduling* dan mampu mengimplementasikannya menggunakan bahasa pemrograman C.
2. Mahasiswa mengerti kekurangan dan kelebihan dari masing-masing *disk scheduling algorithm*.

B. Petunjuk Praktikum

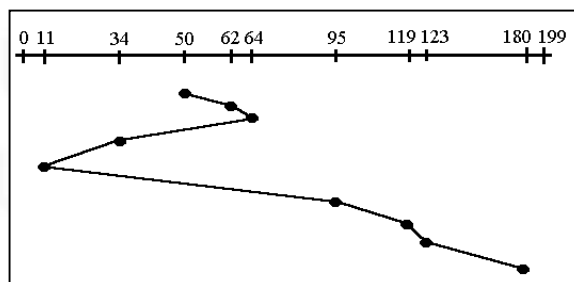
1. Jawaban pertanyaan yang berupa tulisan diketik dalam format .txt.
2. Jawaban pertanyaan yang berupa *source code* diketik dalam format .c.
3. Pengumpulan terakhir dapat dilakukan paling lambat sebelum praktikum selesai.
4. Periksa kembali jawaban Anda, bila telah lengkap kompresilah dengan format: **NIM_NAMA_w11.tar.gz** yang berisikan *text file* dan *source code* atas jawaban untuk Tugas Pendahuluan dan Tugas Praktikum.

C. Materi

Seek time berperan penting dalam sistem operasi. Semua *request* yang dilakukan oleh *device* akan membuat *seek time* bertambah dan memperlambat kerja sistem. *Disk Scheduling Algorithm* digunakan untuk meminimalkan *seek time* dari *device request* yang diterima. Terdapat banyak algoritma *disk scheduling*, diantaranya adalah SSTF dan SCAN.

SSTF (*Shortest Seek Time First*) adalah algoritma *disk scheduling* yang mengutamakan jarak terpendek dari posisi terakhir *head* berada. Pada kasus tertentu, algoritma ini lebih optimal dibandingkan dengan yang lain. Di lain sisi, ada kemungkinan terjadi *starvation* pada bagian yang jauh dari bagian dengan request yang banyak dan berdekatan.

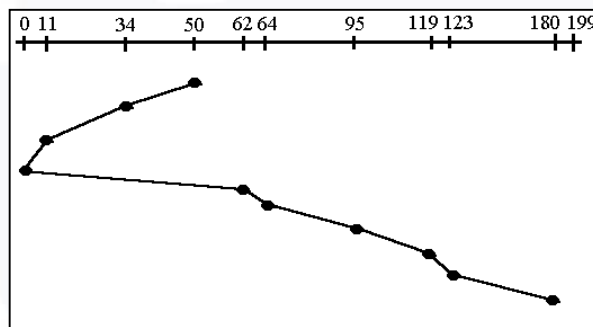
Asumsikan isi *queue*: 95, 180, 34, 119, 11,123, 62, 64 dan posisi *head* berada pada *track* 50. Maka SSTF akan menghasilkan urutan pergerakan *head* seperti gambar berikut ini.



Gambar 1: SSTF Disk Scheduling Algorithm dengan *queue*: 95, 180, 34, 119, 11,123, 62, 64 dan total seek time 236 tracks

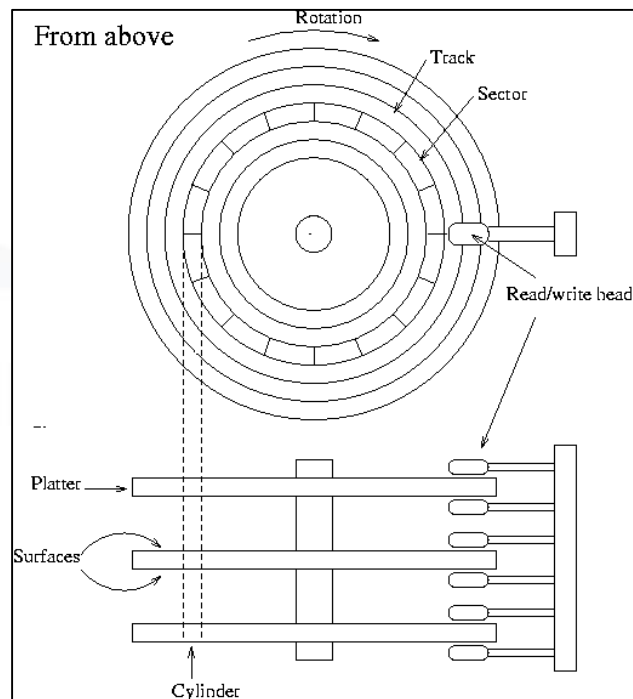
SCAN (disebut juga *elevator*) adalah algoritma *disk scheduling* yang menggerakkan *head* dari ujung yang satu ke ujung yang lain dari *tracks*. Cara kerja dari algoritma ini menyerupai cara kerja *elevator* pada umumnya. *Elevator* akan bergerak turun ke ujung paling dekat. Setelah sampai ke bagian paling bawah, *elevator* akan bergerak naik sampai ke paling atas. Jika ada *request* datang dari bagian yang telah dilewati, maka *request* tersebut tidak akan dilayani sampai *elevator* bergerak turun kembali dan menjumpai bagian tersebut. Algoritma ini lebih optimal dan dapat menghindari *starvation*.

Asumsikan isi *queue*: 95, 180, 34, 119, 11,123, 62, 64, posisi *head* berada pada *track* 50 dan bergerak ke bawah. Maka *SCAN* akan menghasilkan urutan pergerakan *head* seperti gambar berikut ini.



Gambar 2: *SCAN disk scheduling algorithm* dengan *queue*: 95, 180, 34, 119, 11,123, 62, 64 dan total *seek time* 230 tracks

Berikut adalah gambaran akan struktur *hard disk* sebagai penjelasan akan *head*, *sector*, *track*, dan *cylinder*.



Gambar 3: Struktur *hard disk* dilihat dari atas
(<http://www.tldp.org/LDP/sag/html/hd-schematic.png>)

D. Tugas Pendahuluan

Jawablah pertanyaan berikut ke dalam file **jawab.txt**!

1. Sebut dan jelaskan dengan singkat mengenai algoritma *disk scheduling* lainnya selain *SSTF* dan *SCAN* minimal 3!
2. Dari semua algoritma *disk scheduling* yang ada, manakah yang paling baik menurut anda? Jelaskan alasan anda!
3. Algoritma *Shortest Seek Time First* dapat menimbulkan *starvation*. Apa yang dimaksud dengan *starvation* pada *disk scheduling algorithm*? Jelaskan dengan singkat!

E. Tutorial

Buatlah sebuah program dan beri nama **sstf_scan.c**. Program akan mengerjakan simulasi *disk scheduling algorithm* menurut algoritma *SCAN* dan *SSTF*. *Track* terkecil direpresentasikan dengan angka 0, sedangkan *track* terbesar direpresentasikan dengan angka 199. Untuk algoritma *SCAN*, pergerakan pertama dapat diasumsikan keatas atau kebawah terlebih dahulu.

Input

Baris pertama terdiri dari sebuah deretan angka yang merupakan *queue device request*. Baris selanjutnya merupakan posisi dimana *head* berada.

Output

Keluarkan urutan perjalanan *head* dari awal sampai akhir *request* dan total *seek time* untuk masing-masing algoritma (*SSTF* dan *SCAN*).

Contoh:

Input:

95 180 34 119 11 123 62 64
50

Output:

SSTF
62 64 34 11 95 119 123 180
236
SCAN bawah
34 11 62 64 95 119 123 180
230

Catatan:

pada *SCAN*, total *seek time* = $16 + 23 + 11 + 62 + 2 + 31 + 24 + 4 + 57 = 230$.

pada *SCAN*, perjalanan dari 11 menuju 0 (ataupun yang menuju 199) terhitung ke dalam total *seek time*.

Praktikum 12

Device Driver DMA Controller Sederhana

Pokok Bahasan

Pemahaman dan implementasi device driver DMA Controller sederhana



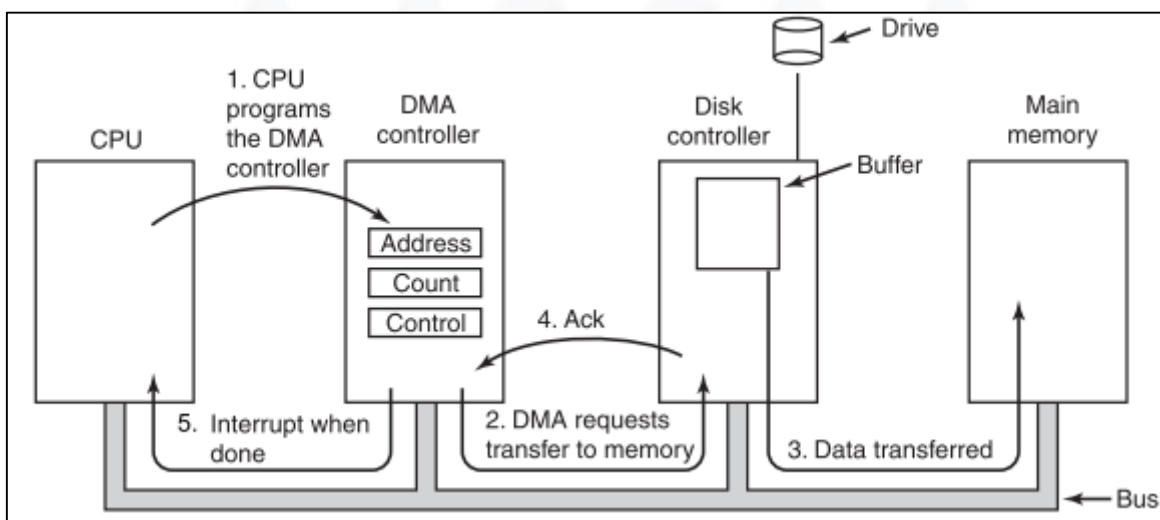
A. Tujuan Praktikum

1. Mahasiswa memahami fungsi dan cara kerja *device driver* sederhana pada umumnya.
2. Mahasiswa memahami cara kerja *DMA* dalam membantu meringankan *workload CPU* (prosesor) dalam proses transfer data.
3. Mahasiswa mampu mengimplementasikan *device driver* sederhana untuk *DMA Controller* dengan menggunakan bahasa pemrograman C.

B. Petunjuk Praktikum

1. Jawaban pertanyaan yang berupa tulisan diketik dalam format .txt.
2. Jawaban pertanyaan yang berupa *source code* diketik dalam format .c.
3. Pengumpulan terakhir dapat dilakukan paling lambat sebelum praktikum selesai.
4. Periksa kembali jawaban Anda, bila telah lengkap kompresilah dengan format: **NIM_NAMA_w12.tar.gz** yang berisikan *text file* dan *source code* atas jawaban untuk Tugas Pendahuluan dan Tugas Praktikum.

C. Materi



Gambar 1: Arsitektur komputer dengan *DMA Controller* untuk proses transfer data

Direct Memory Access (DMA) Controller adalah sebuah komponen komputer yang berfungsi untuk meringankan beban kerja komputer dalam melakukan transfer data.

Perpindahan data dalam jumlah besar dari satu perangkat ke perangkat lainnya merupakan sebuah pekerjaan sederhana yang tidak membutuhkan komputasi. Pada sistem komputer yang tidak dilengkapi perangkat *DMA Controller*, prosesor harus mengerjakan perpindahan data tersebut satu per satu dari alamat sumber ke alamat destinasi. Dengan adanya perangkat *DMA Controller*, prosesor cukup mengatur perangkat untuk melakukan perpindahan data tersebut. Sehingga prosesor dapat kembali mengerjakan pekerjaan lainnya dan *DMA Controller* mengerjakan proses perpindahan data tersebut secara independen dan paralel. Saat proses transfer data sudah selesai maka *DMA Controller* akan mengirimkan sinyal kepada prosesor untuk menginformasikan bahwa proses perpindahan data yang diminta telah selesai. Skema kerja ini detailnya dapat dilihat di Gambar 1 di atas. *DMA Controller* dapat melayani proses perpindahan data dari memori ke perangkat (*output*), perangkat ke memori (*input*), atau *memory-to-memory*.

Device driver adalah komponen perangkat lunak (*software*) yang berfungsi untuk membuat sebuah sistem komputer dapat berkomunikasi dengan sebuah perangkat keras. Perangkat lunak ini pada umumnya diakses dan digunakan oleh kernel sebuah sistem operasi pada saat proses *booting* berlangsung. Selain cara tersebut, beberapa perangkat keras juga menawarkan konsep *plug-and-play* yaitu *device driver* akan terhubung atau dapat diakses oleh sistem operasi pada saat dibutuhkan saja, yaitu saat perangkat keras tersebut terhubung ke sistem komputer. Contoh dari skema ini adalah beberapa perangkat keras di pasaran seperti *mouse*, *printer*, *scanner*, dan lain-lain yang saat ini sudah mengusung konsep *plug-and-play*. Beberapa sistem operasi juga memungkinkan *device driver* dieksekusi dari bagian *user space* sistem operasi tersebut. Beberapa *driver* yang telah terpasang pada sistem operasi dari awal untuk mendukung proses kerja sebuah sistem komputer, tetapi untuk perangkat keras yang baru dan akan dihubungkan ke sistem komputer tersebut, dibutuhkan *driver* spesifik dari perangkat keras baru tersebut agar sistem operasi dapat mengaksesnya. *DMA Controller* sebagai sebuah perangkat keras pada sebuah sistem komputer, tentunya juga membutuhkan *device driver* yang sesuai dengan karakteristiknya agar sistem operasi dapat mengakses dan mengelola *DMA Controller* dengan baik.

D. Tugas Pendahuluan

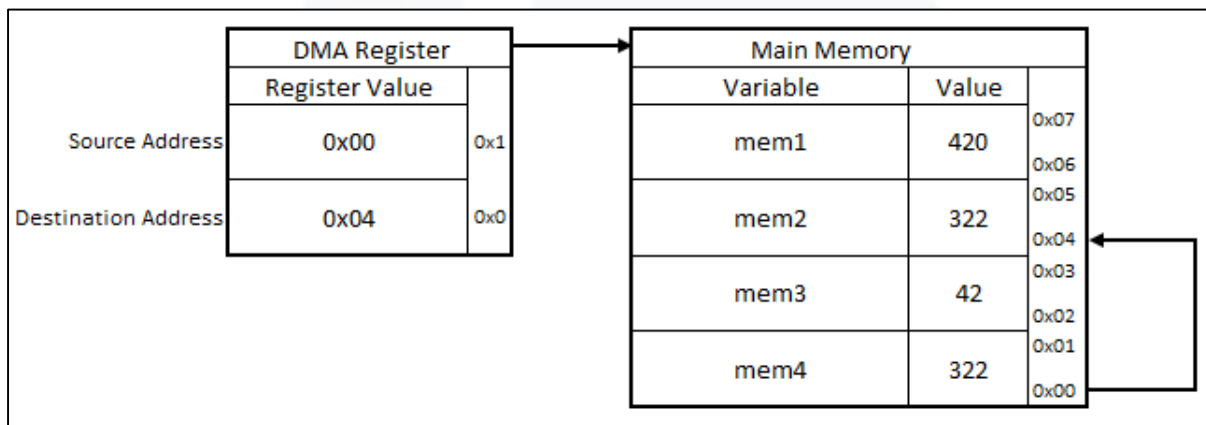
Jawablah pertanyaan berikut ke dalam file **jawab.txt**!

1. Apa kerugian dan keuntungan yang didapatkan dari implementasi fitur DMA pada perangkat komputer? Jelaskan dengan singkat!
2. DMA dapat ditemukan pada berbagai macam perangkat keras, khususnya perangkat I/O. Sebutkan lima perangkat keras yang menggunakan DMA!
3. Jelaskan dengan singkat tentang *device driver* beserta kegunaannya!

E. Tutorial

Buatlah program dengan nama **simple_dma.c**, program ini akan menyimulasikan proses perpindahan data oleh *DMA controller*. Diasumsikan bahwa *DMA Controller* akan secara otomatis melakukan proses perpindahan datanya pada saat alamat tujuan dimasukkan. *Input*

dari program adalah *register address* dan *register value* untuk tiap-tiap *register DMA Controller*, yaitu *source address* dan *destination address*. *Register address* melambangkan bagian mana dari *register DMA* yang akan diisi oleh *register value*, sedangkan *register value* merupakan isi atau nilai untuk register tersebut, yaitu artinya alamat sumber atau alamat tujuan. Variabel yang tersedia untuk perpindahan adalah 4 buah variabel dengan nama dan nilai berturut-turut yaitu **mem1=420**, **mem2=727**, **mem3=42**, dan **mem4=322**. Apabila mendapatkan input berupa -1 -1, program akan selesai dan menampilkan nilai akhir dari *main memory* untuk tiap-tiap variabelnya. Apabila dimasukkan alamat yang tidak valid maka *output* program adalah "INVALID ADDRESS". Penjelasan mengenai variabel, *register*, dan beserta alamatnya dapat dilihat pada gambar di bawah ini.



Gambar 2: Skema kerja DMA Controller sederhana

Batasan dari program adalah pasangan input *register address* dan *register value* yaitu bilangan heksadesimal. Selain itu, 4 buah variabel yang digunakan untuk perpindahan data merupakan 4 buah **signed short** dengan jarak nilai -32768 sampai dengan 32767. DMA diasumsikan akan selalu memindahkan data dengan ukuran 2 byte dalam 1 proses. Tugas utama anda adalah membuat sebuah fungsi (submodul) pada program yang menerima parameter yang dibutuhkan untuk melakukan proses transfer data ini. Fungsi ini kemudian mengembalikan alur kerja ke modul utama (*main*) setelah proses transfer selesai.

Contoh Input

0x0 0x04

0x1 0x00

0x1 0x08

0x2 0x00

0x1 0x02

0x1 0x05

0x0 0x00

-1 -1

Praktikum 13

Device Driver DMA Controller Sederhana

Pokok Bahasan

Pemahaman dan implementasi device driver DMA Controller sederhana



A. Tujuan Praktikum

1. Mahasiswa memahami fungsi dan cara kerja *device driver* sederhana pada umumnya.
2. Mahasiswa memahami cara kerja *DMA* dalam membantu meringankan *workload CPU* (prosesor) dalam proses transfer data.
3. Mahasiswa mampu mengimplementasikan *device driver* sederhana untuk *DMA Controller* dengan menggunakan bahasa pemrograman C.

B. Petunjuk Praktikum

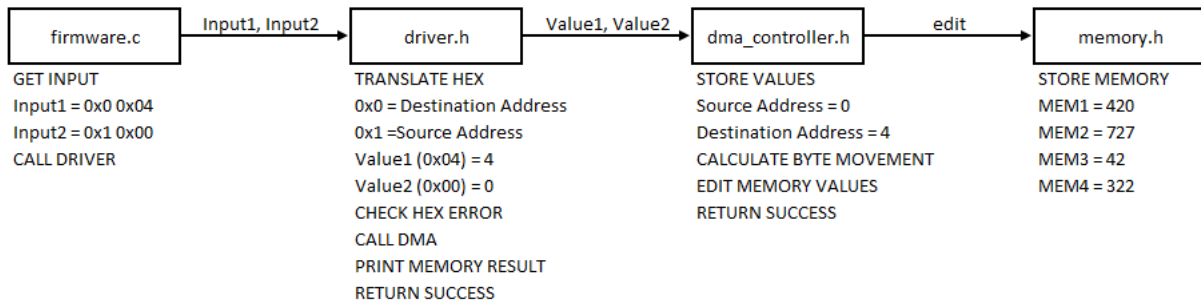
1. Jawaban pertanyaan yang berupa tulisan diketik dalam format .txt.
2. Jawaban pertanyaan yang berupa *source code* diketik dalam format .c.
3. Pengumpulan terakhir dapat dilakukan paling lambat sebelum praktikum selesai.
4. Periksa kembali jawaban Anda, bila telah lengkap kompresilah dengan format: **NIM_NAMA_w12.tar.gz** yang berisikan *text file* dan *source code* atas jawaban untuk Tugas Pendahuluan dan Tugas Praktikum.

C. Tutorial

Ubahlah program **simple_dma.c** kalian dari praktikum sebelumnya agar lebih menyerupai proses kerja DMA sesungguhnya. Partisikan program kalian menjadi 4 bagian, yaitu **firmware.c**, **driver.h**, **dma_controller.h**, dan **memory.h**. **firmware.c** bertugas untuk menerima input dari user dan kemudian memberikannya ke **driver.h**. Kemudian, **driver.h** akan mengubah nilai hexadesimal *register address* untuk menentukan variabel yang akan diubah dan mengubah nilai hexadesimal *register value* untuk disimpan di **dma_controller.h**. **dma_controller.h** akan memiliki 2 buah data, yaitu *source_address* dan *destination_address*, yang mana nilainya ditentukan oleh **driver.h**. Kemudian **dma_controller.h** akan memanipulasi variabel memori di dalam **memory.h** sesuai dengan nilai yang dimilikinya. Keempat program akan dikaitkan dengan cara *include*.

Pada praktikum ini, keempat bagian dari simulasi DMA berperan masing-masing sebagaimana seharusnya pada perangkat aslinya. **firmware.c** berperan sebagai Firmware yang meminta proses dilakukan. **driver.h** bertugas menangkap permintaan dari Firmware untuk diolah oleh DMA, dalam kasus praktikum ini, driver akan mentranslasikan nilai heksadesimal. Setelah permintaan siap diolah, maka **dma_controller.h** akan mengerjakan tugasnya yaitu mengolah perpindahan data sesuai dengan permintaan. **memory.h** berperan sebagai memori utama yang menyimpan data-data, dalam kasus ini yaitu *mem1*, *mem2*, *mem3*, dan *mem4*. Dapat dilihat bahwa dalam

simulasi ini, **firmware.c** hanya bertugas memanggil fungsi satu kali saja, sehingga dapat segera melakukan pekerjaan lain.



Gambar 1: Skema kerja keempat program pecahan

Batasan dari program adalah pasangan input *register address* dan *register value* yaitu bilangan heksadesimal. **dma_controller.h** akan menyimpan source dan destination address yang telah diolah dengan tipe data apapun. Selain itu, **memory.h** akan menyimpan 4 buah variabel yang merupakan 4 buah **signed short** dengan jarak nilai -32768 sampai dengan 32767. DMA diasumsikan akan selalu memindahkan data dengan ukuran 2 byte dalam 1 proses. Tidak boleh ada proses yang dikerjakan di **memory.h** (hanya menyimpan data).

Contoh Input

0x0 0x04
0x1 0x00

0x1 0x08
0x2 0x00

0x1 0x02
0x1 0x05
0x0 0x00

-1 -1

Contoh Output

INVALID ADDRESS
INVALID ADDRESS
mem1 = 420 (0000 0001 1010 0100)
mem2 = 322 (0000 0001 0100 0010)
mem3 = 42 (0000 0000 0010 1010)
mem4 = -23551 (1010 0100 0000 0001)

Penjelasan Contoh Kasus

2 baris input pertama melambangkan 1 buah proses perpindahan DMA. Setelah input diterima, **firmware.c** akan segera mengoper nilai tersebut ke **driver.h**. **driver.h** kemudian akan mengolah nilai heksadesimal tersebut, dan kemudian mengubah nilai *source address* dan *destination address* milik **dma_controller.h** sesuai dengan input (*source* 0x0, *destination* 0x1). Setelah *source address* dan *destination address* terisi, maka **dma_controller.h** akan

melakukan perpindahan, pada kasus ini, 2 byte dari 0x00 (**mem4**) dipindahkan ke alamat 0x04 (**mem2**).

Pada 2 baris input berikutnya, terjadi 2 kali kesalahan input. Kesalahan pertama adalah *register value* yang berisi alamat yang tidak ada di main memory, kesalahan kedua adalah *register address* yang tidak ada di DMA. Kedua input dianulir dan **driver.h** akan menampilkan eror "INVALID ADDRESS".

2 baris input berikutnya hanya mengisi *register value* pada *source address* 0x1 saja, maka proses perpindahan belum dapat dijalankan. Nilai yang disimpan sebagai *register value* adalah nilai yang terakhir dimasukkan, yaitu 0x05. Baru setelah baris berikutnya memasukkan nilai *register value* untuk *destination address*, proses perpindahan dilakukan. Pada kasus ini, 2 byte dari 0x05 (berarti 8 *most-significant bit* dari **mem2** dan 8 *least-significant bit* dari **mem1**) dipindahkan ke alamat 0x00 (**mem4**).

Baris input terakhir berisikan -1 -1 yang merupakan penanda akhir dari input. Setelah menemui input demikian, program segera selesai dan **driver.h** akan mencetak kondisi memori utama terakhir, yakni seluruh nama variabel dan nilainya (dalam desimal dan bit).



Praktikum 14

KUIS

Pokok Bahasan

Kuis mengenai materi dari minggu ke-8 sampai minggu ke-13



Petunjuk

1. Pilih dan kerjakan hanya **satu** dari dua soal yang ada!

Soal

1. Buatlah sebuah program dengan nama **soal01.c** yang mensimulasikan sebuah **Disk Scheduling Algorithm** yaitu **SCAN** dimulai dengan pergerakan **head** keatas ($0 \leq \text{track number} \leq 187$)

Input:

Baris pertama berisi angka **n** yang menjadi panjang **queue track** yang dibutuhkan. Baris selanjutnya berisi **n** angka yang merepresentasikan **track** yang ada pada queue. Baris selanjutnya berisi posisi awal **head** berada.

Output:

Baris pertama berisikan **track** yang dikunjungi oleh **head** secara berurutan. Baris selanjutnya, tampilkan total **seek time** dari head.

Contoh:

Input:

8

118 29 90 59 57 110 6 175

37

Output:

57 59 90 110 118 175 **187** 29 6

331

2. Buatlah sebuah program dengan nama **soal02.c** yang mensimulasikan **Virtual Memory Address Translation**. Perpindahan alamat memori akan dibuat sesuai dengan tabel dibawah ini:

Page Number	Frame Number	Resident Bit
15	1010	0
14	X	1
13	1111	1
12	X	0
11	0001	0
10	X	0
9	1100	1
8	X	1
7	X	0
6	0011	1
5	1001	1
4	0010	1
3	X	0
2	X	0
1	0100	1
0	X	0

Setiap alamat memori terdiri dari **8 digit heksadesimal** dengan **1 most significant digit**nya merupakan **page number** dan **frame number** masing-masing. **7 digit** sisanya merupakan **offset**.

Input:

Program akan terus menerima input sampai **CTRL-C** ditekan. Dalam satu baris berisi sebuah alamat **virtual memory** yaitu **8 digit heksadesimal**.

Output:

Untuk setiap baris input, keluarkan alamat **physical memory** hasil translasi dari **virtual memory** yang telah dimasukkan. Jika hasil translasi tidak ada pada tabel (bertuliskan X) maka tuliskan **PAGE FAULT**.

Contoh:

Input:

0x6feaf373

0x3fcf71cb

0x95e3e5e4

Output:

0011 1111 1110 1010 1111 0011 0111 0011 (0x3feaf373)

PAGE FAULT

1100 0101 1110 0011 1110 0101 1110 0100 (0xc5e3e5e4)